Session 2 - Race conditions
Geavanceerde computerarchitectuur: labosessie

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

February 19, 2026



Year: 2026

Class: MELEC

Instructor:

Contents

1 Context **2**

2 max detection (nested for loop on the CPU) **2**

3 max detection (GPU kernel employing atomic operation) **2**

4 max detection (GPU kernel using reduction) **2**

5 Observations/results **3**

6 Conclusion **4**

A Code **4**

1 Context

Het doel van deze tweede labosessie is het onderzoeken en oplossen van race conditions in een parallelle programmeeromgeving. Een race condition treedt op wanneer meerdere threads gelijktijdig hetzelfde geheugenadres proberen te manipuleren, wat kan leiden tot inconsistente of foutieve resultaten [2]. We hanteren drie verschillende manieren om de maximale waarde in een 1D-array te bepalen: een sequentiële aanpak op de CPU, een aanpak op de GPU met atomaire functies en een aanpak op de GPU die gebruikmaakt van een reductie-algoritme[2]. Vervolgens evalueren we de resultaten van de verschillende implementaties en de invloed van de arraylengte op de uitvoertijd.

2 max detection (nested for loop on the CPU)

De CPU-implementatie dient als referentie. Zo weten we wat de correcte resultaten zijn alsook de prestaties bij een traditionele sequentiële aanpak. Hierbij wordt zoals eerder vermeld een standaard sequentiële aanpak gebruikt, waarbij een (geneste) for-lus elk element in de input-array inspecteert. Omdat de CPU de operaties na elkaar uitvoert op een enkele thread of een beperkt aantal threads, zijn er geen extra synchronisatiestappen nodig om race conditions te voorkomen.

3 max detection (GPU kernel employing atomic operation)

Deze methode maakt gebruik van de ingebouwde functie `atomicMax()` om de maximale waarde te bepalen. Atomaire operaties garanderen dat een commando zonder onderbreking wordt uitgevoerd. Threads krijgen sequentiële toegang tot een gemeenschappelijke bron (het adres waar de maximale waarde wordt bijgehouden), waardoor schrijffouten door gelijktijdige toegang worden vermeden[2]. Hoewel dit de programmeercomplexiteit laag houdt, kan het bij een zeer groot aantal threads leiden tot prestatieverlies door "resource contention" (threads die op de lock moeten wachten)[3].

4 max detection (GPU kernel using reduction)

Het reductie-algoritme is een semi-parallelle aanpak die ideaal is voor inputs met een lengte van 2^N. De complexiteit van dit algoritme is[2]:

$$\mathcal{O}(\log_2(N)) \tag{1}$$

In deze benadering vergelijken threads elementen paren en slaan ze het resultaat op voor de volgende stap. Hierdoor wordt de arraylengte bij elke stap gehalveerd tot men het resultaat vind. Cruciaal hierbij is het gebruik van `_syncthreads()` na elke reductiestap. Dit commando zorgt ervoor dat alle threads in een block het gesynchroniseerd zijn en dat alle geheugentoegangen zichtbaar zijn voor de andere threads voordat de volgende stap begint. Zonder deze synchronisatie ontstaan er data hazards (zoals read-after-write), wat de berekening onbetrouwbaar maakt [3].

5 Observations/results

```
Array size: 1024
Average CPU Time: 0.00234656 ms, Result: 998792
Average GPU Atomic Time: 0.0211866 ms, Result: 998792 [PASSED]
Average GPU Reduction Time: 0.0210694 ms, Result: 998792 [PASSED]

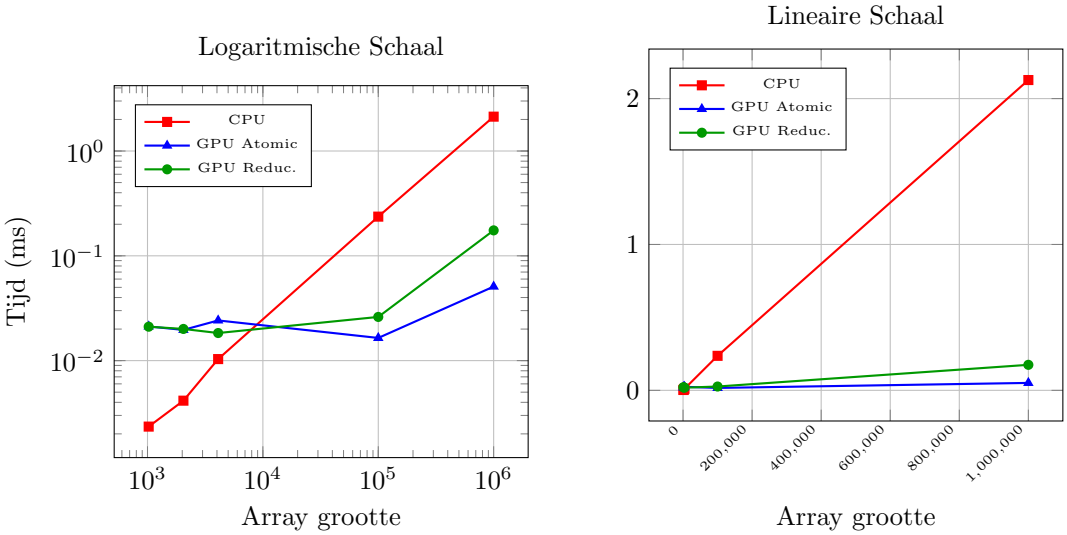
Array size: 2048
Average CPU Time: 0.00414879 ms, Result: 999916
Average GPU Atomic Time: 0.0196403 ms, Result: 999916 [PASSED]
Average GPU Reduction Time: 0.0200566 ms, Result: 999916 [PASSED]

Array size: 4096
Average CPU Time: 0.0103439 ms, Result: 999575
Average GPU Atomic Time: 0.0241766 ms, Result: 999575 [PASSED]
Average GPU Reduction Time: 0.018336 ms, Result: 999575 [PASSED]

Array size: 100000
Average CPU Time: 0.236504 ms, Result: 999979
Average GPU Atomic Time: 0.0164557 ms, Result: 999979 [PASSED]
Average GPU Reduction Time: 0.0261165 ms, Result: 999979 [PASSED]

Array size: 1000000
Average CPU Time: 2.12719 ms, Result: 999998
Average GPU Atomic Time: 0.0508723 ms, Result: 999998 [PASSED]
Average GPU Reduction Time: 0.174897 ms, Result: 999998 [PASSED]
```

Figure 1: Output in terminal



(a) Logaritmische assen

(b) Lineaire assen

Figure 2: Vergelijking van uitvoeringstijden op een logaritmische en lineaire schaal.

Tijdens de experimenten observeren we hoe de uitvoeringstijd schaalt met de arraylengte N. Terwijl de CPU-tijd lineair toeneemt, tonen de GPU-methoden een efficiëntere aanpak bij grotere datasets. Hierbij is het wel opmerkelijk dat de `atomicMax()` aanpak sneler is dan de reductie aanpak. Dit valt waarschijnlijk te wijten aan extra overhead die te maken heeft met het managen van de grote hoeveelheid threads bij reductie.

a. **Schaalbaarheid:** Voor inputs groter dan 2048 elementen moeten we rekening houden met de limieten van één thread block (maximaal 1024 threads/block). Dit vereist het gebruik van meerdere blocks en extra synchronisatie tussen de resultaten van elk block.

b. **Efficiëntie:** De reductiemethode is theoretisch sneller door de logaritmische complexiteit, maar eveneens beheer door middel van `_syncthreads()` om correcte resultaten te garanderen. Hierdoor is de snelheid lager dat wat theoretisch mogelijk is.

c. **Atomics:** Atomaire operaties zijn eenvoudig te implementeren maar kunnen bij extreme parallelliteit een bottleneck vormen door het "locken" van resources [3].

d. **Grid-stride loop**: Wanneer het aantal threads veel kleiner is dan het aantal data-elementen moeten we ervoor zorgen dat alle data verwerkt wordt. Om dit te garanderen kan men een grid-stride loop implementeren [4]. In plaats van één element te verwerken, voert elke thread een lus uit waarin hij zijn ellement verwerkt en dan kijkt of er nog te verwerken elementen zijn (hierbij is de stapgrootte van de lus is gelijk aan het totaal aantal threads in de grid).

e. **Linearisatie:** Om een 2D-array (zoals een matrix) op de GPU te verwerken, maken we gebruik van linearisatie. Hoewel wij de data zien als een tabel met rijen en kolommen, slaat de computer dit op als één lange, platte reeks in het geheugen. In CUDA gebeurt dit standaard in row-major format. Als de matrix een breedte heeft van N elementen, is de formule [5]:

$$GlobalIndex = (i \cdot N) + j \tag{2}$$

6 Conclusion

In dit labo leerden we omgaan met race conditions, een veel voorkomend probleem bij parallel programmeren op de GPU. Atomaire functies bieden een veilige en eenvoudige manier om gedeelde bronnen aan te passen door threads sequentiële toegang te verlenen. Echter, voor maximale performantie bij grote datasets is een reductie-algoritme met synchronisatie via `_syncthreads()` beter. Daarnaast is het correct mappen van data op de threads cruciaal bij het omgaan met grote hoeveelheden data.

A Code

```
1
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5 #include <chrono>
6 #include <climits>
7 #include <cuda_runtime.h>
8
9 #define blockSize 1024
```

```
10
11 // CUDA kernel to add two arrays
12 int max_loop(const int* data, int n) {
13     int max_val = INT_MIN;
14     for (int i = 0; i < n; ++i) {
15         if (data[i] > max_val) {
16             max_val = data[i];
17         }
18     }
19     return max_val;
20 }
21
22 __global__ void max_atomic(int *data, int n, int *max_val){
23      int idx = threadIdx.x + blockIdx.x * blockDim.x;
24      if(idx < n){
25         atomicMax(max_val, data[idx]);
26      }
27 }
28
29 __global__ void max_reduction(int *data, int n, int *max_val){
30     extern __shared__ int sdata[];
31
32     int tid = threadIdx.x;
33     int idx = threadIdx.x + blockIdx.x * blockDim.x;
34     // Load data into shared memory
35     sdata[tid] = (idx < n) ? data[idx] : INT_MIN;
36     __syncthreads();
37     // Reduction in shared memory
38     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
39         if (tid < s) {
40             sdata[tid] = max(sdata[tid], sdata[tid + s]);
41         }
42         __syncthreads();
43     }
44     if (tid == 0) {
45         atomicMax(max_val, sdata[0]);
46     }
47 }
48
49
50 void run(int n){
51 std::cout <<  \nArray size:   << n << std::endl;
52     // Allocate host memory
53     std::vector<int> h_data(n);
54     // Initialize with random numbers
55     for (int i = 0; i < n; ++i) {
56         h_data[i] = rand() % 1000000;
57     }
58
59      //GPU param
60     int *d_data, *d_max_val;
61     cudaMalloc(&d_data, n * sizeof(int));
62     cudaMalloc(&d_max_val, sizeof(int));
63     cudaMemcpy(d_data, h_data.data(), n * sizeof(int), cudaMemcpyHostToDevice);
64     int h_max_val;
65
66 // --- max_loop ---
67     auto start_cpu = std::chrono::high_resolution_clock::now();
68     int cpu_result;
69     int num_runs = 100;
70     for(int i=0; i<num_runs; i++){
71         cpu_result = max_loop(h_data.data(), n);
```

```
72      }
73      auto end_cpu = std::chrono::high_resolution_clock::now();
74      std::chrono::duration<double, std::milli> elapsed_cpu = end_cpu - start_cpu;
75      std::cout <<  Average CPU Time:   << elapsed_cpu.count() / num_runs <<   ms,
        Result:   << cpu_result << std::endl;
76
77      // ---max_atomic---
78      int initial_max = INT_MIN;
79      int gridSize = (n + blockSize - 1) / blockSize;
80      cudaEvent_t start, stop;
81      cudaEventCreate(&start);
82      cudaEventCreate(&stop);
83
84      // single run for correct value
85      cudaMemcpy(d_max_val, &initial_max, sizeof(int), cudaMemcpyHostToDevice);
86      max_atomic<<<gridSize, blockSize>>>(d_data, n, d_max_val);
87      cudaMemcpy(&h_max_val, d_max_val, sizeof(int), cudaMemcpyDeviceToHost);
88
89      // Timing loop (wrong value because of kernel loop, but better time average)
90      cudaMemcpy(d_max_val, &initial_max, sizeof(int), cudaMemcpyHostToDevice); //
        Reset for timing
91      cudaEventRecord(start);
92      for(int i=0; i<num_runs; i++){
93          max_atomic<<<gridSize, blockSize>>>(d_data, n, d_max_val);
94      }
95      cudaEventRecord(stop);
96
97      cudaEventSynchronize(stop);
98      float milliseconds_atomic = 0;
99      cudaEventElapsedTime(&milliseconds_atomic, start, stop);
100
101     std::cout <<  Average GPU Atomic Time:   << milliseconds_atomic / num_runs <<
          ms, Result:   << h_max_val;
102     if (h_max_val == cpu_result) std::cout <<   [PASSED]  << std::endl;
103     else std::cout <<   [FAILED]  << std::endl;
104
105     // ---max_reduction---
106
107     // Single run for correct value
108     cudaMemcpy(d_max_val, &initial_max, sizeof(int), cudaMemcpyHostToDevice);
109     max_reduction<<<gridSize, blockSize, blockSize * sizeof(int)>>>(d_data, n,
        d_max_val);
110     cudaMemcpy(&h_max_val, d_max_val, sizeof(int), cudaMemcpyDeviceToHost);
111
112     // Timing loop (wrong value because of kernel loop, but better time average)
113     cudaEventRecord(start);
114     for(int i=0; i<num_runs; i++){
115         max_reduction<<<gridSize, blockSize, blockSize * sizeof(int)>>>(d_data, n,
         d_max_val);
116     }
117     cudaEventRecord(stop);
118     cudaEventSynchronize(stop);
119     float milliseconds_reduction = 0;
120     cudaEventElapsedTime(&milliseconds_reduction, start, stop);
121     std::cout <<  Average GPU Reduction Time:   << milliseconds_reduction /
        num_runs <<   ms, Result:   << h_max_val;
122     if (h_max_val == cpu_result) std::cout <<   [PASSED]  << std::endl;
123     else std::cout <<   [FAILED]  << std::endl;
124
125     // Cleanup
126     cudaFree(d_data);
127     cudaFree(d_max_val);
```

```
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
}

int main() {
    srand(time(NULL));
    // Test with different sizes
    run(1024);
    run(2048);
    run(4096);
    // Larger sizes
    run(100000);
    run(1000000);
}
```

Listing 1: CUDA Kernel for detection of the maximum value in a array

References

[1] Feys T. Gent (2026) *CUDA_C_Programming_Guide-exercise_2*, Lab notes.

[2] Feys T. Gent (2026) *session-2*, Lab notes.

[3] Lenovo België (2023) *Parallellisatie: Wat is parallellisatie & de verschillende soorten?*, Glossary. Beschikbaar op: `https://www.lenovo.com/be/nl/glossary/parallelization/`

[4] Minnaar A. (2019) *CUDA Grid-Stride Loops: What if you Have More Data Than Threads?*, Blog post. Beschikbaar op: `https://alexminnaar.com/2019/08/02/grid-stride-loops.html`

[5] Veitner (2025) *Indexing in CUDA*, Blog post. Beschikbaar op: `https://veitner.bearblog.dev/indexing-in-cuda/`